Create VS Code extension for autonomous coding *(using OpenClaw Gateway, Ollama with DeepSeek-R1:8b approximately 5.2 GBs)*

Table of Contents

Create VS Code extension for autonomous coding *(using OpenClaw Gateway, Ollama with DeepSeek-R1:8b approximately 5.2 GBs)*

- what is open claw actually?
- can i configure it to use deepseek and can it be used in VS Code for vibe coding ?
- can you create a vs code extension for openclaw to be used in vs code for coding agent?
- i would like openclaw running on my local machine with deepseek on my local machine and a vs code extension with permissions and setting options like openai's codex extension for vs code.
- okay do it
  - Run it
- "Local model: Ollama can run DeepSeek open-weight models locally, and OpenClaw supports Ollama’s native API. That means no paid API key or per-token charge, though the model’s speed and quality depend on your computer’s memory and hardware."
- tell me what each of Ollama, DeepSeek-R1:8b, OpenClaw are actually and will do actually in my case? How will latest information will be fetched if this setup is locally hosted?
- will i be able to train my deepseek-r1:8b locally downloaded model? will it be improved as i use it?
- so how to download and setup everything and is it a possibility for my vs code extension above to use vs code builtin coding agents precisely in this order: (copilot, codex, claude agent, cursor, gemini and if every fails use my own OpenClaw Gateway with Ollama and DeepSeek-R1:8b setup as fallback)?

User prompt 1 of 9:

0.1. what is open claw actually?

Assist:
OpenClaw is a free, open-source autonomous AI agent that runs locally on your computer and can carry out tasks using large language models. You use it through messaging apps like WhatsApp, Telegram, Discord, Signal, or WhatsApp, and it can do actions such as reading/writing files and running tools to complete work.

Read More

- https://milvus.io/blog/openclaw-formerly-clawdbot-moltbot-explained-a-complete-guide-to-the-autonomous-ai-agent.md
- https://en.wikipedia.org/wiki/OpenClaw

User prompt 2 of 9 - 02/11/2026, 04:26:40:

0.1. can i configure it to use deepseek and can it be used in VS Code for vibe coding ?

GPT-6 Luna:
Yes. **OpenClaw supports DeepSeek**: install its provider plugin, then run onboarding and choose DeepSeek:

```
openclaw plugins install @openclaw/deepseek-provider
openclaw onboard --auth-choice deepseek-api-key
```

You'll need a DeepSeek API key. Onboarding prompts for it and sets a default model; you can check available models with `openclaw models list --provider deepseek`. [1,2]

For vibe coding in VS Code: OpenClaw is mainly a personal assistant you interact with through chat or its own interface—not a VS Code coding extension. You can still use it alongside VS Code, but for AI that works directly in your editor (reading project context and helping edit code), use a VS Code coding extension or a terminal coding agent integrated with VS Code. DeepSeek documents integrations with coding tools such as OpenCode; OpenClaw's documented setup is through its chat interface or terminal. [2,3]

Read More
[1] openclaw/docs/providers/deepseek.md at main · openclaw/openclaw — https://github.com/openclaw/openclaw/blob/main/docs/providers/deepseek.md
[2] DeepSeek · OpenClaw — https://docs.openclaw.ai/providers/deepseek
[3] Integrate with AI Tools | DeepSeek API Docs — https://api-docs.deepseek.com/guides/coding_agents/

User prompt 3 of 9 - 02/11/2026, 04:28:43:

0.1. can you create a vs code extension for openclaw to be used in vs code for coding agent?

GPT-6 Luna:

Yes. I can generate the extension's source code here, though I can't directly install files into your VS Code workspace.

A sensible first version would add an **OpenClaw sidebar chat** with commands to send your selected code or current file as context, ask for edits, and review proposed changes before applying them. It could connect to your local OpenClaw Gateway using its OpenAI-compatible `/v1/chat/completions` endpoint; the Gateway supports that endpoint and uses its configured auth token. [1]

I'd make it **approval-first**: the agent can suggest code changes, but the extension shows a diff and waits for you to accept before editing files. That's a safer fit for a coding agent than letting it modify your

workspace automatically.

Read More
[1] Gateway runbook · OpenClaw — https://docs.openclaw.ai/gateway

---

User prompt 4 of 9 - 02/11/2026, 04:36:04:

0.1. i would like openclaw running on my local machine with deepseek on my local machine and a vs code extension with permissions and setting options like openai's codex extension for vs code.

And if DeepSeek doesn't provide API Key for free use then use a free ai which provides this free api key to use for ai based coding agent, or use php curl to get the ai chat and get code from there for free usage without paid api key.

GPT-6 Luna:

Yes—this is a workable setup. I'd build it as a **local VS Code coding agent** with OpenClaw handling the agent and tools, and Ollama serving a model on your machine.

- **Local model:** Ollama can run DeepSeek open-weight models locally, and OpenClaw supports Ollama's native API. That means no paid API key or per-token charge, though the model's speed and quality depend on your computer's memory and hardware. [1,2]
- **DeepSeek API:** DeepSeek's hosted API is billed by token usage, so it isn't a dependable free-API route. [3,4]
- **VS Code extension:** I'd give it a chat sidebar, selected-code/current-file context, workspace-aware requests, model and connection settings, and diff previews before applying edits. For permissions, it should ask before file changes and terminal commands, with configurable approval rules.
- **Important security detail:** OpenClaw's compatible chat endpoint is disabled by default. Its docs also say a Gateway token grants operator-level access, so the extension should connect only to the local Gateway, keep that token in VS Code's secret storage, and avoid exposing the Gateway to the internet. [5]

I wouldn't scrape DeepSeek's web chat with PHP cURL to avoid API billing; that's an unreliable workaround, not a supported API. For free local use, the clean route is Ollama plus a downloaded model. I can generate the installable extension project source next, with **manual approvals on by default**.

Read More
[1] openclaw/docs/providers/ollama.md at main · openclaw/openclaw — https://github.com/openclaw/openclaw/blob/main/docs/providers/ollama.md
[2] Ollama setup · OpenClaw — https://docs.openclaw.ai/providers/ollama/setup
[3] Models & Pricing | DeepSeek API Docs — https://api-docs.deepseek.com/quick_start/pricing/
[4] Your First API Call | DeepSeek API Docs — https://api-docs.deepseek.com/
[5] OpenAI chat completions · OpenClaw — https://docs.openclaw.ai/gateway/openai-http-api

User prompt 5 of 9 - 02/11/2026, 04:39:48:

0.1. okay do it

GPT-6 Luna:

Here's a **working first MVP**: a VS Code sidebar chat that sends your prompt and selected code to your local OpenClaw Gateway. It stores the Gateway token in VS Code's secret storage and requires confirmation before inserting the assistant's reply into your editor.

This version **does not run terminal commands or autonomously edit files**. That's deliberate: OpenClaw's HTTP endpoint uses Gateway authentication and should be treated as an owner-level access surface, so keep it on localhost and don't expose it publicly. The chat-completions endpoint is disabled by default; enable it in OpenClaw first. [1]

Create a folder named `openclaw-vscode` and add these files:

`package.json`

```
{
  "name": "openclaw-vscode",
  "displayName": "OpenClaw Coding Chat",
  "description": "A local VS Code coding chat for OpenClaw",
  "version": "0.0.1",
  "publisher": "local",
  "engines": {
    "vscode": "^1.90.0"
  },
  "categories": ["Other"],
  "activationEvents": ["onView:openclaw.chat"],
  "main": "./out/extension.js",
  "contributes": {
    "views": {
      "explorer": [
        {
          "id": "openclaw.chat",
          "name": "OpenClaw"
        }
      ]
    },
    "commands": [
      {
        "command": "openclaw.setToken",
        "title": "OpenClaw: Set Gateway Token"
      }
    ],
    "configuration": {
      "title": "OpenClaw",
      "properties": {
        "openclaw.endpoint": {
```

```
          "type": "string",
          "default": "http://127.0.0.1:18789",
          "description": "Local OpenClaw Gateway URL"
        },
        "openclaw.agent": {
          "type": "string",
          "default": "openclaw/default",
          "description": "OpenClaw agent target"
        },
        "openclaw.confirmBeforeInsert": {
          "type": "boolean",
          "default": true,
          "description": "Ask before inserting the assistant response into the editor"
        }
      }
    }
  },
  "scripts": {
    "compile": "tsc -p .",
    "vscode:prepublish": "npm run compile"
  },
  "devDependencies": {
    "@types/node": "^20.0.0",
    "@types/vscode": "^1.90.0",
    "typescript": "^5.4.0"
  }
}
```

tsconfig.json

```
{
  "compilerOptions": {
    "module": "commonjs",
    "target": "ES2022",
    "outDir": "out",
    "lib": ["ES2022", "DOM"],
    "sourceMap": true,
    "rootDir": "src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  },
  "exclude": ["node_modules", ".vscode-test"]
}
```

src/extension.ts

```
import * as vscode from "vscode";

const TOKEN_KEY = "openclaw.gatewayToken";
```

```
export function activate(context: vscode.ExtensionContext) {
  context.subscriptions.push(
    vscode.commands.registerCommand("openclaw.setToken", async () => {
      const token = await vscode.window.showInputBox({
        title: "OpenClaw Gateway Token",
        prompt: "Enter the token for your local OpenClaw Gateway",
        password: true,
        ignoreFocusOut: true
      });

      if (token) {
        await context.secrets.store(TOKEN_KEY, token);
        vscode.window.showInformationMessage("OpenClaw token saved securely.");
      }
    }),

    vscode.window.registerWebviewViewProvider(
      "openclaw.chat",
      new ChatViewProvider(context),
      { webviewOptions: { retainContextWhenHidden: true } }
    )
  );
}

class ChatViewProvider implements vscode.WebviewViewProvider {
  constructor(private readonly context: vscode.ExtensionContext) {}

  resolveWebviewView(view: vscode.WebviewView) {
    view.webview.options = { enableScripts: true };
    view.webview.html = this.html();

    view.webview.onDidReceiveMessage(async (message) => {
      if (message.type === "send") {
        await this.sendPrompt(view, String(message.text ?? ""));
      } else if (message.type === "insert") {
        await this.insertReply(String(message.text ?? ""));
      } else if (message.type === "setToken") {
        await vscode.commands.executeCommand("openclaw.setToken");
      }
    });
  }

  private async sendPrompt(view: vscode.WebviewView, prompt: string) {
    const token = await this.context.secrets.get(TOKEN_KEY);
    if (!token) {
      view.webview.postMessage({
        type: "error",
        text: "Set your Gateway token first: run “OpenClaw: Set Gateway Token” from the
Command Palette."
      });
      return;
    }

    const config = vscode.workspace.getConfiguration("openclaw");
    const endpoint = config.get<string>("endpoint", "http://127.0.0.1:18789")
```

```
      .replace(/\/+$/, "");
    const agent = config.get<string>("agent", "openclaw/default");

    const editor = vscode.window.activeTextEditor;
    const selectedText = editor
      ? editor.document.getText(editor.selection)
      : "";

    const contextText = selectedText
      ? `\n\nSelected code (language:
${editor?.document.languageId}):\n\`\`\`\n${selectedText}\n\`\`\``
      : "";

    const userMessage = `${prompt}${contextText}`;

    view.webview.postMessage({ type: "status", text: "Thinking…" });

    try {
      const response = await fetch(`${endpoint}/v1/chat/completions`, {
        method: "POST",
        headers: {
          "Authorization": `Bearer ${token}`,
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          model: agent,
          user: "vscode-openclaw",
          messages: [
            {
              role: "system",
              content:
                "You are a coding assistant. Explain proposed changes clearly. " +
                "Do not claim to have edited files or run commands."
            },
            { role: "user", content: userMessage }
          ],
          stream: false
        })
      });

      const data = await response.json() as {
        choices?: Array<{ message?: { content?: string } }>;
        error?: { message?: string };
      };

      if (!response.ok) {
        throw new Error(data.error?.message ?? `HTTP ${response.status}`);
      }

      const answer = data.choices?.[0]?.message?.content;
      if (!answer) throw new Error("OpenClaw returned an empty response.");

      view.webview.postMessage({ type: "reply", text: answer });
    } catch (error) {
      view.webview.postMessage({
```

```
        type: "error",
        text: `Request failed: ${error instanceof Error ? error.message : String(error)}`
      });
    }
  }

  private async insertReply(text: string) {
    const config = vscode.workspace.getConfiguration("openclaw");
    if (config.get<boolean>("confirmBeforeInsert", true)) {
      const choice = await vscode.window.showWarningMessage(
        "Insert the assistant's entire reply at the current cursor?",
        { modal: true },
        "Insert"
      );
      if (choice !== "Insert") return;
    }

    const editor = vscode.window.activeTextEditor;
    if (!editor) {
      vscode.window.showErrorMessage("Open a file and place the cursor first.");
      return;
    }

    await editor.edit((edit) => edit.replace(editor.selection, text));
  }

  private html() {
    return `<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<style>
  body { font-family: var(--vscode-font-family); padding: 8px; }
  textarea { box-sizing: border-box; width: 100%; min-height: 90px; }
  button { margin: 6px 4px 6px 0; }
  #reply { white-space: pre-wrap; overflow-wrap: anywhere; margin-top: 12px; }
  #status { opacity: .8; }
</style>
</head>
<body>
  <textarea id="prompt" placeholder="Ask about your code…"></textarea>
  <div>
    <button id="send">Ask OpenClaw</button>
    <button id="token">Set token</button>
    <button id="insert" hidden>Insert reply…</button>
  </div>
  <div id="status"></div>
  <pre id="reply"></pre>
<script>
  const vscode = acquireVsCodeApi();
  const prompt = document.getElementById("prompt");
  const reply = document.getElementById("reply");
  const status = document.getElementById("status");
  const insert = document.getElementById("insert");
```

```
  let latestReply = "";

  document.getElementById("send").onclick = () => {
    const text = prompt.value.trim();
    if (text) vscode.postMessage({ type: "send", text });
  };
  document.getElementById("token").onclick = () =>
    vscode.postMessage({ type: "setToken" });
  insert.onclick = () =>
    vscode.postMessage({ type: "insert", text: latestReply });

  window.addEventListener("message", event => {
    const message = event.data;
    if (message.type === "status") status.textContent = message.text;
    if (message.type === "reply") {
      latestReply = message.text;
      reply.textContent = message.text;
      status.textContent = "";
      insert.hidden = false;
    }
    if (message.type === "error") {
      status.textContent = message.text;
      insert.hidden = true;
    }
  });
</script>
</body>
</html>`;
  }
}

export function deactivate() {}
```

0.0.1. Run it

1. Enable OpenClaw's `/v1/chat/completions` endpoint in its configuration:

   ```
   {
     gateway: {
       http: {
         endpoints: {
           chatCompletions: { enabled: true }
         }
       }
     }
   }
   ```

   Ensure OpenClaw is configured to use your local Ollama model and keep the Gateway bound to localhost. The endpoint routes requests to an OpenClaw agent, such as `openclaw/default`. [1]

2. In the extension folder, run:

```
npm install
npm run compile
```

3. Open the folder in VS Code, press **F5** to launch an Extension Development Host, then run **OpenClaw: Set Gateway Token** from the Command Palette.

This MVP sends selected code only when you ask a question and offers an explicit, confirmed insert. It does **not** yet provide Codex-style tool permissions, file-write approvals, or terminal-command approvals; those should be added as separate, explicit permissioned actions rather than letting the model execute arbitrary commands.

Read More
[1] OpenAI chat completions · OpenClaw — https://docs.openclaw.ai/gateway/openai-http-api

---

User prompt 6 of 9 - 02/11/2026, 04:53:10:
you said previously:

0.1. "Local model: Ollama can run DeepSeek open-weight models locally, and OpenClaw supports Ollama's native API. That means no paid API key or per-token charge, though the model's speed and quality depend on your computer's memory and hardware."

how to achieve this in easy steps?

GPT-6 Luna:

The easiest route is **Ollama → download a DeepSeek model → connect OpenClaw to Ollama**. The model runs on your computer, so you don't need a DeepSeek API key or pay per token. You do need internet to download Ollama and the model initially.

1. **Install Ollama** from its official download page, then open a terminal and check it's running:

```
ollama --version
```

2. **Download and try a DeepSeek model.** A reasonable starting point is the 8B DeepSeek-R1 model:

```
ollama run deepseek-r1:8b
```

The first run downloads the model; after that, it runs locally. You can type a prompt to test it, then exit with `Ctrl+D`. The Ollama model library also offers smaller and larger variants; bigger models generally need more memory. [1,2]

3. **Connect OpenClaw to Ollama.** Run:

   ```
   openclaw onboard
   ```

   Choose **Ollama**, then **Local only**. Select the local Ollama host and the model you downloaded. OpenClaw's setup checks available models; if the model isn't offered or fails its tool-support check, choose an eligible model shown by onboarding instead. [3,4]

4. **Check the connection and choose the default model:**

   ```
   openclaw models list --provider ollama
   openclaw models status
   ```

   If needed, set the default using the exact model name shown in your list, for example:

   ```
   openclaw models set ollama/deepseek-r1:8b
   ```

   [5,6]

5. **Use the VS Code extension** from the previous message with OpenClaw running. It sends requests to OpenClaw, which uses the configured Ollama model. You don't configure a DeepSeek API key in the extension.

One detail: OpenClaw uses Ollama's **native API**, not Ollama's OpenAI-compatible `/v1` URL. The onboarding flow handles that connection for you. [3,7]

Read More
[1] GitHub - ollama/ollama: Get up and running with Kimi, GLM, MiniMax, ... — https://github.com/ollama/ollama
[2] deepseek-r1 — https://ollama.com/library/deepseek-r1
[3] Ollama · OpenClaw — https://docs.openclaw.ai/providers/ollama
[4] Ollama setup · OpenClaw — https://docs.openclaw.ai/providers/ollama/setup
[5] Models CLI · OpenClaw — https://docs.openclaw.ai/concepts/models
[6] Models · OpenClaw — https://docs.openclaw.ai/cli/models
[7] openclaw/docs/providers/ollama.md at main · openclaw/openclaw — https://github.com/openclaw/openclaw/blob/main/docs/providers/ollama.md

---

User prompt 7 of 9 - 02/11/2026, 04:59:15:

0.1. tell me what each of Ollama, DeepSeek-R1:8b, OpenClaw are actually and will do actually in my case? How will latest information will be fetched if this setup is locally hosted?

GPT-6 Luna:

Think of the three parts as **runtime, model, and agent**:

- **Ollama** is the local app that downloads and runs AI models on your computer. It loads the model and provides a local interface for other software to send it prompts.
- **DeepSeek-R1:8b** is the model—the "brain" that reads your prompt and generates a response. The `8b` version is a smaller, distilled model; its download is about 5.2 GB. It can answer from what it learned during training, but its built-in knowledge isn't continuously updated. [1,2]
- **OpenClaw** is the agent layer. It connects to the model and can coordinate tasks and tools—for example, pass it your coding request and, if configured, let it use web search or fetch a webpage. It doesn't automatically make the model itself current.

For your setup, a coding request would go from the VS Code extension to OpenClaw, then to Ollama, which runs DeepSeek-R1 locally and returns its answer. **The extension we sketched so far sends your prompt and selected code and displays the answer; it does not yet call OpenClaw's web-search tools or autonomously edit your project.**

For current information, the distinction is:

- **No web tool configured:** DeepSeek answers from its model knowledge and the context you provide. It won't know about new releases or today's documentation unless you provide that information.
- **Web search/fetch configured in OpenClaw:** OpenClaw can retrieve current pages and pass relevant results to the local model to summarize. Your computer still needs an internet connection, and the retrieved pages go over the network. OpenClaw lists key-free search options such as DuckDuckGo and Ollama Web Search, as well as self-hosted SearXNG; a search provider must be explicitly selected/configured rather than assumed to be active. [3]

So you can keep **the model's inference local** and still fetch current web information, but web search is a separate capability. If you want your VS Code coding agent to research up-to-date libraries or documentation, that needs to be enabled in OpenClaw and then wired into the extension's agent requests.

Read More
[1] deepseek-r1 — https://ollama.com/library/deepseek-r1
[2] deepseek-r1:8b — https://ollama.com/library/deepseek-r1:8b
[3] Web search · OpenClaw — https://docs.openclaw.ai/tools/web

---

User prompt 8 of 9 - 02/11/2026, 05:02:42:

0.1. will i be able to train my deepseek-r1:8b locally downloaded model? will it be improved as i use it?

GPT-6 Luna:

No—DeepSeek-R1:8b won't train itself or get smarter just because you use it. Its model weights stay the same unless you deliberately fine-tune them.

What *can* improve with use is **OpenClaw's memory**, which is separate from training. OpenClaw can save preferences, project notes, and useful facts in local files and bring them into later conversations. That can make the assistant feel more personalized, but it doesn't change DeepSeek's underlying abilities or teach it new coding skills automatically. [1]

You can fine-tune a model locally with a curated set of examples, using a separate training tool such as Unsloth, then load the resulting model into Ollama. That's a technical training process—not something Ollama or OpenClaw does automatically—and you'd need to test whether the fine-tuned version actually performs better. [2,3]

For your coding setup, I'd start with OpenClaw memory and project instructions rather than fine-tuning. If your goal is to teach it about a codebase or internal docs, giving it those files as searchable context is usually more practical than changing the model weights.

Read More
[1] Memory overview · OpenClaw — https://docs.openclaw.ai/concepts/memory
[2] Tutorial: How to Finetune Llama-3 and Use In Ollama | Unsloth ... — https://unsloth.ai/docs/get-started/fine-tuning-llms-guide/tutorial-how-to-finetune-llama-3-and-use-in-ollama
[3] deepseek-ai/DeepSeek-R1 · Hugging Face — https://huggingface.co/deepseek-ai/DeepSeek-R1

User prompt 9 of 9 - 02/11/2026, 05:14:59:

0.1. so how to download and setup everything and is it a possibility for my vs code extension above to use vs code builtin coding agents precisely in this order: (copilot, codex, claude agent, cursor, gemini and if every fails use my own OpenClaw Gateway with Ollama and DeepSeek-R1:8b setup as fallback)?

GPT-6 Luna:

Yes, the fallback sequence is possible—but not by making one extension silently take over the built-in Copilot, Codex, Claude, Cursor, and Gemini chat panels. VS Code has APIs for extensions to use language models and contribute agent tools, but each coding agent has its own interface and permissions. There isn't a common VS Code API that lets your extension drive all those agents as if they were one interchangeable service. [1,2,3]

The practical approach is to have your extension call each agent through a **supported CLI or SDK**, in your chosen order, then call OpenClaw last. Copilot CLI, for example, supports programmatic prompts and

specific tool-permission flags; Cursor CLI also supports scripted prompts, but its print mode has access to file-writing and shell tools—so its permissions need careful handling. The other providers likewise need their own supported integration and working sign-in. [4,5,6,7]

A safe sequence would be:

1. Try Copilot, then Codex, Claude, Cursor, and Gemini.
2. Move to the next one only if the previous provider is unavailable or returns a defined failure—not merely because the extension dislikes its answer.
3. If all fail, call your local OpenClaw Gateway using the Ollama model.
4. Keep a human approval step for proposed changes and commands. Run each attempt in a separate Git worktree or otherwise isolate its changes; agents can modify files before reporting an error, and blindly starting the next one could stack conflicting edits.

One distinction: using **VS Code's Language Model API** to ask Copilot for a response is not the same as invoking Copilot's full coding agent. The API requires user consent and can hit account limits; it doesn't automatically inherit the agent's complete UI, tools, and approval flow. [1,2]

For the local fallback, here's the setup:

1. Install Ollama from its official download page, then download the model:

   ```
   ollama pull deepseek-r1:8b
   ollama run deepseek-r1:8b
   ```

   The model download is about 5.2 GB. The `run` command also lets you test it; exit with `Ctrl+D`. [8]

2. Install OpenClaw. On macOS/Linux:

   ```
   curl -fsSL https://openclaw.ai/install.sh | bash
   ```

   On Windows PowerShell:

   ```
   iwr -useb https://openclaw.ai/install.ps1 | iex
   ```

   Then run:

   ```
   openclaw onboard
   ```

   Choose **Ollama** → **Local only**, and select the local model. [9,10,11]

3. Check that OpenClaw sees the model and the Gateway is running:

```
openclaw models list --provider ollama
openclaw gateway status
```

OpenClaw's Ollama setup checks that a model can handle tools and has at least a 16K context window. If it doesn't accept `deepseek-r1:8b` for agent use, the onboarding flow may require a different compatible local model. [11]

4. For our VS Code extension to reach that fallback through OpenClaw, enable OpenClaw's chat-completions endpoint and configure the Gateway token in the extension. OpenClaw itself connects to Ollama through Ollama's native API; the extension's connection to OpenClaw is a separate hop. Keep the Gateway on localhost and don't expose its token or port publicly. [12,13]

So: **the local OpenClaw fallback is straightforward; the ordered multi-provider router is a larger extension feature.** It should integrate with the agents' supported CLIs/SDKs—not pretend their private VS Code panels share a common API.

Read More

[1] Language Model API | Visual Studio Code Extension API — https://code.visualstudio.com/api/extension-guides/ai/language-model
[2] AI language models in VS Code — https://code.visualstudio.com/docs/agent-customization/language-models
[3] VS Code API | Visual Studio Code Extension API — https://code.visualstudio.com/api/references/vscode-api
[4] GitHub Copilot CLI programmatic reference - GitHub Docs — https://docs.github.com/en/copilot/reference/copilot-cli-reference/cli-programmatic-reference
[5] Running GitHub Copilot CLI programmatically - GitHub Docs — https://docs.github.com/en/copilot/how-tos/copilot-cli/automate-copilot-cli/run-cli-programmatically
[6] Using Agent in CLI | Cursor Docs — https://cursor.com/docs/cli/using
[7] Cursor CLI | Cursor Docs — https://cursor.com/docs/cli/overview
[8] deepseek-r1:8b — https://ollama.com/library/deepseek-r1:8b
[9] openclaw/docs/install/index.md at main · openclaw/openclaw — https://github.com/openclaw/openclaw/blob/main/docs/install/index.md
[10] Install · OpenClaw — https://docs.openclaw.ai/install
[11] Ollama setup · OpenClaw — https://docs.openclaw.ai/providers/ollama/setup
[12] openclaw/docs/providers/ollama.md at main · openclaw/openclaw — https://github.com/openclaw/openclaw/blob/main/docs/providers/ollama.md
[13] Ollama · OpenClaw — https://docs.openclaw.ai/providers/ollama